COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CpE21S3
DATE PERFORMED:	08/01/2026	**DATE SUBMITTED:**	08/01/2026
NAME:	Wayne Kaisen C. Paraiso	**INSTRUCTOR:**	Engr. Jimlord Quejado

Hands-on Activity 4.1 Stacks

1. PROCEDURE OUTPUT

ILO A

ILOA > Paraiso_Seatwork_4_2.cpp > pop()

```cpp
#include <iostream>


const size_t maxCap = 100;
int stack[maxCap];
int top = -1, newData, elements;

bool isFull();
bool isEmpty();
void push();
void pop();
void Top();
void displayAll();

int main(){
    int choice;

    std::cout<<"Enter the number of maximum elements for the new stack (max = 100): ";
    std::cin >> elements;
    elements--;

    while(true){

        std::cout << "Stack Operations: "<<std::endl;
        std::cout << "1. Push\n2. Pop\n3. Top\n4. isEmpty\n5. isFull\n6. displayAll"<<std::endl
        std::cin >> choice;

        switch(choice){
            case 1: push();
            break;
            case 2: pop();
            break;
            case 3: Top();
```

```
34              break;
35              case 4: std::cout<< isEmpty()<< std::endl;
36              break;
37              case 5: std::cout << isFull()<<std::endl;
38              break;
39              case 6: displayAll();
40              break;
41              default: std::cout<< "Invalid Choice"<<std::endl;
42              break;
43          }
44      }
45
46      return 0;
47
48  }
49
50  bool isEmpty(){
51      if(top == -1){
52          std::cout<<"Stack Underflow"<<std::endl;
53          return true;
54      };
55      return false;
56      }
57
58  bool isFull(){
59      if(top >= elements){
60          std::cout<<"Stack Overflow"<<std::endl;
61          return true;
```

```
62      }
63      return false;
64  }
65
66  void push(){
67
68      if(isFull()) return;
69
70      std::cout << "What value do you want to add to the stack?: ";
71      std::cin >> newData;
72      stack[++top] = newData;
73  }
74
75  void pop(){
76      if(isEmpty()) return;
77
78      std::cout<<"Popping: " << stack[top] << std::endl;
79
80      top--;
81  }
82
83  void Top(){
84      if(isEmpty()) return;
85
86      std::cout << "The top element is: "<< stack[top] << std::endl;
87  }
88
89  void displayAll(){
90      if(isEmpty()) return;
91
92      std::cout<< "The elements inside the stack are: "<< std::endl;
93       for(int i = top; i >= 0; i--){
94           std::cout << stack[i] << std::endl;
95       }
96   }
```

```
Enter the number of maximum elements for the new stack (max = 100): 5
Stack Operations:
1. Push
2. Pop
3. Top
4. isEmpty
5. isFull
6. displayAll
```

```
What value do you want to add to the stack?: 5
```

```
The elements inside the stack are:
5
```

```
Popping: 5
```

```
Stack Underflow
```

```
6. displayAll
4
Stack Underflow
1
```

ILO B

ILOB2 > main.cpp > ...

```cpp
#include <iostream>
#include "listStack.h"

int main(){


    std::cout<<"------ Pushing 'I' 'C' 'P' 'E' 'P' ------\n";

    push('I');
    push('C');
    push('P');
    push('E');
    push('P');

    Top<char>();

    std::cout<<"------ Inside the Stack ------\n";
    traverseStack<char>();


    std::cout<<"\n------ popping 'P' 'E' 'P' 'C' 'I' ------\n";
    while(!isEmpty<char>()){
        std::cout<<"popping: "<<pop<char>()<<std::endl;
    }

    pop<char>();

    return 0;
}
```

ILOB2 > C listStack.h > ...

```cpp
#ifndef LISTSTACK_H
#define LISTSTACK_H

template <typename T>
class Node{
    public:
        T data;
        Node<T>* next = nullptr;


};

template <typename T>
Node<T>* head = nullptr;

template <typename T>
void push(T newData){
    Node<T>* newNode = new Node<T>;
    newNode->data = newData;
    newNode->next = head<T>;
    head<T> = newNode;
}

template <typename T>
bool isEmpty(){
    return head<T> == nullptr;
}


template <typename T>
T pop(){
    if(isEmpty<T>()){
        std::cout<<"Stack Underflow"<<std::endl;
```

```
34      }
35
36      Node<T>* temp = head<T>;
37      T tempData = temp->data;
38
39      head<T> = head<T>->next;
40
41      delete temp;
42
43      return tempData;
44  }
45
46  template <typename T>
47  void Top(){
48      if(isEmpty<T>()) return;
49
50      std::cout<<"The top is: " << head<T>->data <<std::endl;
51  }
52
53  template <typename T>
54  void traverseStack(){
55      if(head<T> == nullptr){
56          std::cout<<"The stack is empty"<<std::endl;
57      }
58      while(head<T> != nullptr){
59          std::cout<< head<T>->data;
60          if(head<T>->next != nullptr){
61              std::cout<<"->";
62          }
63          head<T> = head<T>->next;
64      }
65
66  }
67
68  #endif
```

```
------ Pushing 'I' 'C' 'P' 'E' 'P' ------
The top is: P
------ Inside the Stack ------
P->E->P->C->I
------ popping 'P' 'E' 'P' 'C' 'I' ------
Stack Underflow
```

Code Analysis:
Summary

Array-based stack: Uses a fixed global array with a top index and a user-set capacity, offering push, pop, peek, and display operations through a menu loop. isEmpty()/isFull() correctly guard against invalid operations, push adds at stack[++top], pop just decrements top without clearing the value, and displayAll() correctly prints elements in top-to-bottom order. The logic works correctly overall, with the only real quirk being that the menu loop has no way to exit.

Linked list-based stack: Uses a templated Node<T> and a templated global head<T>, so each data type gets its own independent stack via the same shared functions, with push inserting new nodes at the front in O(1). However, pop<T>() and traverseStack<T>() print an "empty" message when the stack is empty but fail to actually return afterward, causing real bugs: pop<T>() will crash from a null-pointer dereference if called on an empty stack, and traverseStack<T>() destructively advances head<T> to nullptr while printing, wiping out the stack and leaking its nodes. This is demonstrated in main.cpp, where calling traverseStack empties the stack early, causing the later pop loop to be skipped entirely and the final manual pop call to crash.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Array-based Stack:

ILOC1 > main.cpp > main()

```cpp
#include <iostream>
#include <string>

const int maxCap = 100;
char stack[maxCap];
int top = -1;

bool isEmpty() {
    return top == -1;
}

void push(char symbol) {
    top++;
    stack[top] = symbol;
}

char pop() {
    char symbol = stack[top];
    top--;
    return symbol;
}

bool isMatch(char opening, char closing) {
    if (opening == '(' && closing == ')') return true;
    if (opening == '{' && closing == '}') return true;
    if (opening == '[' && closing == ']') return true;
    return false;
}

int main() {
    std::string expression;
    std::cout << "Enter Expression: ";
    std::cin >> expression;

    bool balanced = true;

    for (size_t i = 0; i < expression.length(); i++) {
```

```
        char ch = expression[i];

        if (ch == '(' || ch == '{' || ch == '[') {
            push(ch);
        }
        else if (ch == ')' || ch == '}' || ch == ']') {
            if (isEmpty()) {
                balanced = false;
                break;
            }
            char opening = pop();
            if (!isMatch(opening, ch)) {
                balanced = false;
                break;
            }
        }
    }

    if (!isEmpty()) {
        balanced = false;
    }

    if (balanced) {
        std::cout << "Balanced" << std::endl;
    } else {
        std::cout << "Unbalanced" << std::endl;
    }

    return 0;
}
```

```
Enter Expression: (A+B)+(C-D)
Balanced
```

```
Enter Expression: ((A+B)+(C-D)
Unbalanced
```

```
Enter Expression: ((A+B)+[C-D])
Balanced
```

```
Enter Expression: ((A+B]+[C-D]}
Unbalanced
```

Linked List-based Stack:

ILOC2 > main.cpp > main()

```cpp
#include <iostream>
#include <string>
#include "singlyStack.h"

int main(){
    std::string expression;
    std::cout << "Enter Expression: ";
    std::cin >> expression;

    if(isBalanced(expression))
        std::cout << "Balanced" << std::endl;
    else
        std::cout << "Unbalanced" << std::endl;
}
```

ILOC2 > C singlyStack.h > head<T>

```
1  #ifndef LISTSTACK_H
2  #define LISTSTACK_H
3
4  #include <iostream>
5  #include <string>
6
7  template <typename T>
8  class Node{
9      public:
10         T data;
11         Node<T>* next = nullptr;
12
13
14 };
15
16 template <typename T>
17 Node<T>* head = nullptr;
18
19 template <typename T>
20 void push(T newData){
21     Node<T>* newNode = new Node<T>;
22     newNode->data = newData;
23     newNode->next = head<T>;
24     head<T> = newNode;
25 }
26
27 template <typename T>
28 bool isEmpty(){
29     return head<T> == nullptr;
30 }
31
32
33 template <typename T>
34 T pop(){
35     if(isEmpty<T>()){
36         std::cout<<"Stack Underflow"<<std::endl;
37     }
```

```
38
39      Node<T>* temp = head<T>;
40      T tempData = temp->data;
41
42      head<T> = head<T>->next;
43
44      delete temp;
45
46      return tempData;
47  }
48
49  template <typename T>
50  void Top(){
51      if(isEmpty<T>()) return;
52
53      std::cout<<"The top is: " << head<T>->data <<std::endl;
54  }
55
56  template <typename T>
57  void traverseStack(){
58      if(head<T> == nullptr){
59          std::cout<<"The stack is empty"<<std::endl;
60      }
61      while(head<T> != nullptr){
62          std::cout<< head<T>->data;
63          if(head<T>->next != nullptr){
64              std::cout<<"->";
65          }
66          head<T> = head<T>->next;
67      }
68
69
70  }
71
72  bool isMatch(char opening, char closing){
```

```
    if(opening == '(' && closing == ')') return true;
    if(opening == '{' && closing == '}') return true;
    if(opening == '[' && closing == ']') return true;
    return false;
}

bool isBalanced(const std::string& expr){
    for(size_t i = 0; i < expr.length(); i++){
        char ch = expr[i];

        if(ch == '(' || ch == '{' || ch == '['){
            push<char>(ch);
        }
        else if(ch == ')' || ch == '}' || ch == ']'){
            if(isEmpty<char>()){
                return false;
            }
            char opening = pop<char>();
            if(!isMatch(opening, ch)){
                return false;
            }
        }
    }

    return isEmpty<char>();
}

#endif
```

```
Enter Expression: (A+B)+(C-D)
Balanced
```

```
Enter Expression: ((A+B)+(C-D)
Unbalanced
```

```
Enter Expression: ((A+B)+[C-D])
Balanced
```

```
Enter Expression: ((A+B]+[C-D]}
Unbalanced
```

Code Analysis:
Array-based stack: Uses a fixed global char array with a top index, where push() increments top and stores the character, and pop() returns stack[top] before decrementing (both without bounds checking, unlike before). isEmpty() and isMatch() work as previously discussed—checking for an empty stack and matching bracket pairs, respectively. The new logic here is in main(): it reads a string expression, pushes any opening bracket encountered, and on a closing bracket, fails immediately if the stack is empty or if the popped opening bracket doesn't match; after scanning the whole string, it also fails if the stack isn't empty (unclosed brackets remain), correctly implementing balanced-bracket checking.

Linked list-based stack: The templated Node<T>, head<T>, push, isEmpty, pop, Top, and traverseStack functions work as previously discussed, including the same bugs (no early return on empty in pop/traverseStack, risking a null-pointer crash and destructive traversal). The new additions are isMatch(), which checks if an opening/closing bracket pair matches, and isBalanced(), which pushes opening brackets, pops and validates against closing brackets (failing early on mismatch or premature empty stack), and returns whether the stack is empty at the end—implementing the same balanced-bracket logic as the array version, but reusable via a separate function rather than inline in main(). main.cpp simply reads the expression and prints "Balanced" or "Unbalanced" based on isBalanced()'s result.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Claude AI**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar Correction, Summarization, Code Assistance**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **In Grammar Correction, I have a draft that I sent to Claude AI to check and correct the grammar. In summarization, I also had a draft about the code analysis and summarized it using Claude AI, I asked Claude for Ideas on how to implement, ask what errors I had that I couldn't see and the bugs I couldn't see also**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.